Hands-on No. : 02

Topic : Inhertiance_Polymorphism_Abstraction

Date : 11-12-25

1.Employee Payroll Management System

In modern corporate organizations, employees are categorized into different types such as Permanent, Contract, and Intern, each having distinct salary computation rules. To ensure scalability, maintainability, and code reusability, the HR department requires an automated payroll system that leverages object-oriented principles. This system models a real-world enterprise payroll scenario using inheritance and polymorphism, eliminating redundant code while accommodating specialized salary structures across employee categories.

System Overview

Base Class Name: Employee
The Employee class represents the common structure and behavior of all employees in the organization.

Derived Classes:

- PermanentEmployee – Represents full-time employees with additional benefits
- ContractEmployee – Represents temporary workers with deductions
- Intern – Represents trainees receiving fixed stipends

🔹 Base Class: Employee

Attributes (Protected Data Members)

Attribute	Type	Description
empId	String	Unique employee identifier
empName	String	Employee's full name
baseSalary	double	Base salary amount before adjustments

Constructors

Constructor	Description
Employee(String empId, String empName, double baseSalary)	Initializes employee with ID, name, and base salary.

Public Methods

Method	Return Type	Description
getEmpId()	String	Retrieves the employee ID.
getEmpName()	String	Retrieves the employee name.
getBaseSalary()	double	Retrieves the base salary.
calculateSalary()	double	Computes and returns the final salary (to be overridden by subclasses).
toString()	String	Returns a formatted string representation of the employee details.

- **Derived Class: PermanentEmployee**

Represents full-time employees entitled to performance-based bonuses.

Additional Attributes

Attribute	Type	Description
bonus	double	Additional bonus amount

Constructors

Constructor	Description
PermanentEmployee(String empId, String empName, double baseSalary, double bonus)	Initializes permanent employee with bonus details.

Overridden Methods

Method	Return Type	Description
calculateSalary()	double	Returns baseSalary + bonus.

SmartCliff

🔹 Derived Class: ContractEmployee

Represents temporary employees subject to tax and service charge deductions.

Additional Attributes

Attribute	Type	Description
taxRate	double	Tax percentage (e.g., 0.10 for 10%)
serviceCharge	double	Fixed service charge amount

Constructors

Constructor	Description
ContractEmployee(String empId, String empName, double baseSalary, double taxRate, double serviceCharge)	Initializes contract employee with deduction parameters.

Overridden Methods

Method	Return Type	Description
calculateSalary()	double	Returns baseSalary - (baseSalary * taxRate) - serviceCharge.

🔹 Derived Class: Intern

Represents trainee employees who receive a fixed stipend instead of a variable salary.

Additional Attributes

Attribute	Type	Description
stipend	double	Fixed monthly stipend

Constructors

Constructor	Description
Intern(String empId, String empName, double stipend)	Initializes intern with fixed stipend amount.

Overridden Methods

SmartCliff

Method	Return Type	Description
calculateSalary()	double	Returns the fixed stipend amount.

- **Functional Requirements**

ID	Requirement	Description
FR1	Employee Registration	System must allow creating employees of different types with appropriate attributes.
FR2	Salary Calculation – Permanent	System should compute salary for permanent employees by adding bonus to base salary.
FR3	Salary Calculation – Contract	System must compute salary for contract employees by deducting tax and service charges from base salary.
FR4	Salary Calculation – Intern	System must return fixed stipend amount for interns.
FR5	Polymorphic Behavior	System should support polymorphic method invocation (e.g., Employee e = new PermanentEmployee()).
FR6	Display Employee Details	System must provide formatted output showing employee information and computed salary.

2. Digital Online Payment System

In modern fintech applications, organizations must support multiple payment methods such as Credit Card, UPI, and NetBanking to accommodate diverse customer preferences. While each payment method implements distinct validation and processing logic, the overall transaction workflow—validate, process, and generate receipt—remains consistent across all methods. To ensure consistency, maintainability, and extensibility, the system requires an abstraction layer that enforces a unified payment workflow while allowing specialized implementations for each payment mode. This models real-world payment gateway architectures used in enterprise financial applications.

System Overview

Abstract Class Name: Payment
The Payment class defines the common structure and workflow for all payment transactions. It enforces mandatory validation and processing steps through abstract methods while providing a standardized receipt generation mechanism applicable to all payment types.

Concrete Subclasses:

- CreditCardPayment – Handles card-based transactions
- UPIPayment – Manages Unified Payments Interface transactions
- NetBankingPayment – Processes online banking transfers

🔹 Abstract Class: Payment

Attributes (Protected Data Members)

Attribute	Type	Description
transactionId	String	Unique transaction identifier
amount	double	Transaction amount
customerName	String	Name of the customer initiating payment
paymentStatus	String	Current status of payment (default: "Pending")

Constructors

Constructor	Description
Payment(String transactionId, double amount, String customerName)	Initializes payment with transaction details and default status.

Abstract Methods (Must be Implemented by Subclasses)

Method	Return Type	Description
validatePayment()	boolean	Validates payment credentials and requirements specific to the payment method.
processPayment()	boolean	Executes the payment transaction using method-specific processing logic.

Concrete Methods (Common Implementation)

Method	Return Type	Description
executeTransaction()	void	Template method that orchestrates the payment workflow: validate → process → receipt.

Method	Return Type	Description
generateReceipt()	void	Displays a standardized transaction receipt with transaction details and status.
getTransactionId()	String	Retrieves the transaction ID.
getAmount()	double	Retrieves the transaction amount.
getPaymentStatus()	String	Retrieves the current payment status.
setPaymentStatus(String status)	void	Updates the payment status.

- **Concrete Class: CreditCardPayment**

Represents credit/debit card-based payment transactions.

Additional Attributes

Attribute	Type	Description
cardNumber	String	Credit/debit card number
cvv	String	Card verification value
expiryDate	String	Card expiration date (MM/YY format)

Constructors

Constructor	Description
CreditCardPayment(String transactionId, double amount, String customerName, String cardNumber, String cvv, String expiryDate)	Initializes credit card payment with card details.

Implemented Methods

Method	Return Type	Description
validatePayment()	boolean	Validates card number format, CVV length, and expiry date; returns true if valid.
processPayment()	boolean	Simulates card authorization and fund deduction; returns true if successful.

SmartCliff

🔹 Concrete Class: UPIPayment

Represents Unified Payments Interface transactions.

Additional Attributes

Attribute	Type	Description
upiId	String	UPI identifier (e.g., user@bank)
upiPin	String	UPI transaction PIN

Constructors

Constructor	Description
UPIPayment(String transactionId, double amount, String customerName, String upiId, String upiPin)	Initializes UPI payment with UPI credentials.

Implemented Methods

Method	Return Type	Description
validatePayment()	boolean	Validates UPI ID format and PIN; returns true if valid.
processPayment()	boolean	Simulates UPI request approval and instant fund transfer; returns true if successful.

🔹 Concrete Class: NetBankingPayment

Represents online banking payment transactions.

Additional Attributes

Attribute	Type	Description
bankName	String	Name of the bank
accountNumber	String	Customer's bank account number
ifscCode	String	Bank's IFSC code

Constructors

SmartCliff

Constructor	Description
NetBankingPayment(String transactionId, double amount, String customerName, String bankName, String accountNumber, String ifscCode)	Initializes net banking payment with bank details.

Implemented Methods

Method	Return Type	Description
validatePayment()	boolean	Validates bank account number and IFSC code format; returns true if valid.
processPayment()	boolean	Simulates bank authentication and fund transfer through banking gateway; returns true if successful.

- **Functional Requirements**

ID	Requirement	Description
FR1	Payment Method Selection	System must support multiple payment modes (Credit Card, UPI, NetBanking).
FR2	Payment Validation	System must validate payment credentials specific to each payment method before processing.
FR3	Payment Processing	System must execute payment transactions using method-specific processing logic.
FR4	Transaction Workflow Enforcement	System must ensure all payments follow the standard workflow: validate → process → generate receipt.
FR5	Receipt Generation	System must generate a standardized receipt displaying transaction details and status.
FR6	Polymorphic Payment Handling	System should support dynamic method dispatch (e.g., Payment p = new UPIPayment()).
FR7	Status Tracking	System must track and update payment status throughout the transaction lifecycle.

- **Transaction Workflow (Template Method)**

```
executeTransaction() {
    1. if (validatePayment()) {
        2. if (processPayment()) {
            3. setPaymentStatus("Success")
          } else {
            3. setPaymentStatus("Failed")
          }
       } else {
         2. setPaymentStatus("Validation Failed")
       }
    4. generateReceipt()
}
```

SmartCliff